



Tecnologie & Proof of Concept

Capitolato C6 — Second Brain

The Bitless • Gruppo 18 • A.A. 2025/2026

[INSERIRE DATA]

Università degli Studi di Padova — Ingegneria del Software

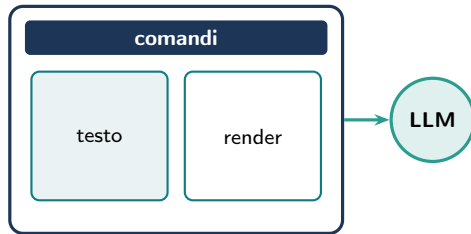
Second Brain

Visione

Un editor **Markdown** affiancato a un **LLM**: un secondo cervello per scrivere, migliorare e fare brainstorming.

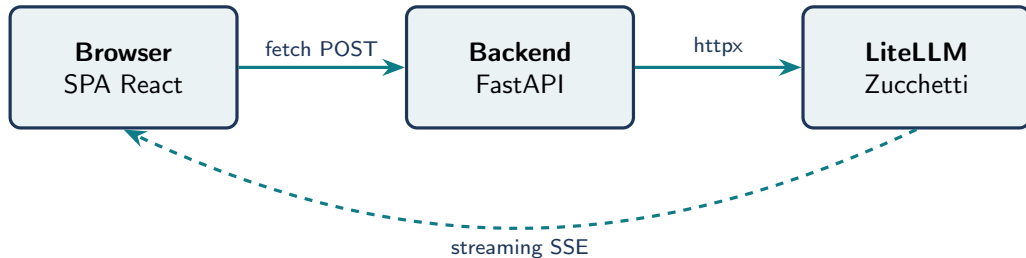
Requisiti obbligatori

- Editor Markdown + rendering live
- Accesso a un LLM sul testo
- Riassunto, riscrittura, traduzione
- Critica “Sei cappelli” di De Bono
- Distant Writing (testo da prompt)
- Salvataggio e lettura note



editor a 2 pannelli + accesso LLM

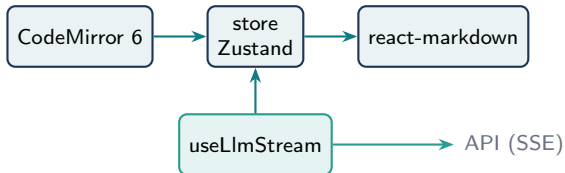
Architettura



- Streaming **end-to-end** con effetto macchina da scrivere
- Backend **stateless**: nessuna logica di dominio lato server

Tecnologie — Frontend

- **React 19 + TypeScript + Vite** — ecosistema maturo, tipi statici, dev server rapido
- **CodeMirror 6** — editor Markdown leggero (~100 KB), undo/redo e highlighting nativi
- **react-markdown + plugin** — pipeline Markdown→AST, anti-XSS, tabelle e LaTeX
- **Zustand** — stato globale senza boilerplate, re-render minimi
- **Tailwind v4 + shadcn/ui** — componenti modificabili, no lock-in, tema chiaro/scuro



Editor — CodeMirror 6 vs alternative

textarea HTML

Semplice, ma niente highlighting Markdown né undo/redo strutturato. Accessibilità da costruire.

Monaco (VS Code)

Potentissimo ma pesante (vari MB), pensato per l'IDE: eccessivo per Markdown.

CodeMirror 6 ✓

Leggero e modulare: highlighting, undo/redo e accessibilità nativi, API tipizzata.

Il giusto compromesso tra potenza e leggerezza per il nostro caso d'uso.

Tecnologie — Backend

- **Python 3.12 + FastAPI** — async nativo per lo streaming LLM, OpenAPI autogenerato
- **Pydantic v2** — validazione runtime, errori 422 espliciti, base per i tipi TypeScript
- **httpx async** — streaming nativo e propagazione dell'annullamento fino al provider
- **Astrazione “LLMClient”** — interfaccia astratta sopra il provider, per testabilità ed estensioni

Backend — FastAPI vs Node.js/Express

Node.js + Express

Ecosistema JS unico FE+BE, ma lo streaming LLM richiede gestire a mano async e backpressure; validazione e OpenAPI da aggiungere con librerie esterne.

FastAPI ✓

Async nativo per stream lunghi senza bloccare; validazione Pydantic e OpenAPI integrate; dall'OpenAPI generiamo i tipi TypeScript del frontend.

Il cuore del progetto è lo streaming LLM: FastAPI lo gestisce nativamente.

Accesso all'LLM — astrazione vs chiamata diretta

Chiamata diretta al provider

Meno codice subito, ma le route restano accoppiate al provider concreto: cambiarlo o aggiungerne uno impone di riscriverle, e i test richiedono l'LLM reale.

LLMClient astratto + LiteLLMClient ✓

Le route dipendono dall'interfaccia, non dal provider: cambiarlo = modifica di config; testabile con mock pytest senza chiamare l'LLM.

Oggi provider unico (LiteLLM Zucchetti); l'astrazione tiene aperta la porta a estensioni future.

Infrastruttura & Qualità

Infrastruttura

- Docker Compose — ambiente riproducibile in un comando
- Mono-repo — frontend + backend, CI unificata

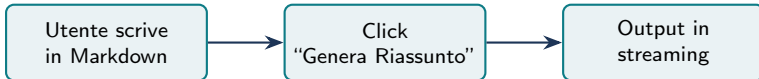
Qualità

- GitHub Actions — CI: lint, type check, test, build
- Test: Vitest + Playwright (FE), pytest (BE)
- openapi-typescript — tipi TS dal backend, zero disallineamenti

Proof of Concept — obiettivo

Obiettivo

Validare la pipeline di **streaming LLM end-to-end** con una sola funzione: il **riassunto**.



Le altre funzioni AI (traduzione, riscrittura, Sei Cappelli, Distant Writing) differiscono **solo per il prompt**: l'architettura resta identica.

Proof of Concept — cosa abbiamo validato

- ✓ Streaming fluido (effetto macchina da scrivere), UI mai bloccata
- ✓ Interruzione propagata su tutta la catena (client → server → provider)
- ✓ Errori del provider gestiti con messaggio chiaro, testo dell'editor preservato
- ✓ Chiave API mai esposta al client
- ✓ Ambiente riproducibile da zero con `docker compose up`

Contatti

The Bitless

`thebitlessgroup.swe@gmail.com`

`https://thebitless.live/`